



MÔN HỌC: MÁY HỌC TRONG BẢO MẬT MẠNG VÀ HỆ THỐNG

HỆ THỐNG LLM ĐA TÁC TỬ Y KHOA NHẪM NÂNG CAO ĐỘ CHÍNH XÁC THỰC TẾ TRÊN MEDQA-USMLE

Báo cáo giữa kỳ

Giảng viên hướng dẫn: Lê Kim Hùng

Nhóm thực hiện: Nhóm 01

Thành viên:

- Nguyễn Đình Thanh - 250202053
- Ngô Văn Tú - 250202057
- Lê Viết Hưng - 250202040
- Trần Quốc Phong - 250202051

Mục lục

1	Giới thiệu	2
2	Kiến trúc hệ thống	3
3	Thiết kế hệ thống	6
3.1	Các biến thể: V0-V5	6
3.2	Thiết kế RAG	10
3.3	Thiết kế bộ nhớ	11
3.4	Thiết kế bộ kiểm chứng	12
4	Thiết lập thực nghiệm	13
4.1	Cấu trúc mã nguồn	13
4.2	Mô hình và prompt	13
4.3	Phần cứng / thiết lập API	14
5	Kết quả đánh giá	15
5.1	Bảng chính (main board): 600 câu đầu của tập test, qwen2.5:7b	15
5.2	So sánh với baseline (so sánh ghép cặp)	15
5.3	Phân tích Thắng/Thua/Hòa	15
5.4	Chi phí và độ trễ	16
6	Phân tích lỗi	16
7	Thảo luận	17
8	Hạn chế	17
9	Kết luận	18

Tóm tắt

Đây là một nghiên cứu về **thiết kế hệ thống**: khi ghép nhiều thành phần (truy hồi, phân vai đa tác tử, bộ nhớ dài hạn, bộ kiểm chứng) vào một pipeline LLM, thành phần nào thực sự làm tăng độ chính xác, và thành phần nào chỉ thêm chi phí mà không thêm gì? Nhóm dùng MedQA-USMLE làm testbed - một bộ dữ liệu trắc nghiệm đòi hỏi lập luận nhiều bước - để trả lời câu hỏi đó một cách có kiểm soát, thay vì nhắm tới một hệ thống sẵn sàng triển khai lâm sàng. Nhóm **thiết kế một kiến trúc đa tác tử dạng module**: một **thang biến thể** (ablation ladder) gồm sáu biến thể (V0 LLM trực tiếp $\rightarrow$ V5 hệ đa tác tử đầy đủ với bộ nhớ dài hạn và bộ kiểm chứng), trong đó mỗi biến thể chỉ là cùng một đồ thị **StateGraph** của LangGraph, bật/tắt đúng một thành phần theo một cấu hình định sẵn - nên mọi chênh lệch độ chính xác giữa hai biến thể liền nhau quy thẳng được về thành phần đó. Nhóm **triển khai một quy trình đánh giá tái lập được**: một bộ đo dùng chung cho cả sáu biến thể (độ chính xác kèm khoảng tin cậy bootstrap, tỷ lệ phản hồi không hợp lệ, phân tích Thắng/Thua/Hòa, và kiểm định ghép cặp McNemar), với **một baseline LLM trực tiếp** (V0, không truy hồi, không đa tác tử) làm mốc **so sánh** cho mọi biến thể khác. Trên 600 câu đầu của tập test, Nhóm **phân tích đóng góp của từng thành phần** qua so sánh ghép cặp giữa các biến thể liền kề: riêng truy hồi (V1) không cải thiện độ chính xác có ý nghĩa thống kê so với V0 (-0.8 điểm), trong khi tách cấu trúc lập luận thành bốn tác tử chuyên trách (V2) là cải thiện có ý nghĩa thống kê đầu tiên trong toàn khảo sát ($+5.5$ điểm, McNemar $p=0.0064$) và bộ nhớ dài hạn (V3) giữ được mức cải thiện đó so với V0 ($+5.2$ điểm, $p=0.0118$); nhưng không biến thể nào thêm vào trên V2 - kể cả V3 - cải thiện có ý nghĩa thống kê so với chính V2. Nhóm ghi lại đầy đủ kiến trúc, prompt và cấu hình để mọi kết quả trên đều tái lập được từ mã nguồn.

1 Giới thiệu

Ghép nhiều thành phần vào một pipeline LLM - truy hồi (RAG), phân vai đa tác tử, bộ nhớ dài hạn, một bước kiểm chứng - là chuyện dễ làm; do xem *từng thành phần đó thực sự đóng góp bao nhiêu* lại khó hơn nhiều, vì các thành phần thường được thêm cùng lúc, không tách riêng được. MedAgents [9] cung cấp bằng chứng ban đầu rằng phối hợp đa tác tử theo dạng hội đồng chuyên gia - các tác tử độc lập đề xuất phân tích, sau đó đi đến đồng thuận - cải thiện lập luận y khoa zero-shot trên MedQA-USMLE; nhưng hệ thống đó gộp đồng thời nhiều thành phần (truy hồi, hội đồng, nhiều vòng thảo luận), khiến khó tách biệt đóng góp riêng của từng yếu tố. Đây là một bài toán **thiết kế hệ thống**, không phải bài toán triển khai lâm sàng: Nhóm dùng MedQA-USMLE làm testbed - một bộ dữ liệu trắc nghiệm đòi hỏi lập luận nhiều bước, có nhãn đúng khách quan để chấm điểm tự động - để trả lời câu hỏi *bắt đầu từ một LLM thuần, mỗi thành phần kiến trúc - truy hồi tri thức (RAG), phối hợp đa tác tử, bộ nhớ dài hạn, bước kiểm chứng (verifier) - đóng góp bao nhiêu vào độ chính xác, và với chi phí (độ trễ, token) nào?*

Bài toán. Đầu vào là một câu hỏi trắc nghiệm MedQA-USMLE [1] (một tình huống lâm sàng kèm bốn lựa chọn A-D). Đầu ra bắt buộc là *đúng một* lựa chọn cuối cùng cộng với một giải thích ngắn (≤ 30 từ).

Cách tiếp cận. Nhóm không thực hiện tách rời sáu hệ thống riêng biệt. Thay vào đó, mỗi biến thể là *cùng một* bộ khung LangGraph [4], chỉ bật/tắt các nút (node) khác nhau theo một cấu hình định sẵn, và mỗi bậc chỉ bổ sung *đúng một* thành phần so với bậc trước: $V0 \rightarrow V1$ thêm truy hồi agentic (RAG), $V1 \rightarrow V2$ tách một tác tử duy nhất thành bốn nút chuyên trách - hiểu ca bệnh, tìm kiếm, lập luận, quyết định (Mục 3.1), $V2 \rightarrow V3$ thêm bộ nhớ dài hạn cho nút quyết định, còn $V3 \rightarrow V4$ và $V3 \rightarrow V5$ mỗi nhánh thêm đúng một trong hai thiết kế bộ kiểm chứng khác nhau lên

trên V3. Thiết kế bốn nút này khác hẳn hội đồng nhiều chuyên gia kiểu MedAgents mà Nhóm đã thử trước khi ổn định trên kiến trúc hiện tại (§3.1). Vì một bộ đo duy nhất chấm cả sáu biến thể theo cùng một bộ đánh giá, mọi chênh lệch điểm số đều so sánh được trực tiếp.

Đóng góp của nhóm nghiên cứu.

1. **Thiết kế một kiến trúc đa tác tử dạng module**, điều khiển bằng cấu hình (V0-V5) - việc khám phá ban đầu lấy cảm hứng từ MedAgents [9], nhưng kiến trúc hiện tại là một thiết kế bốn nút riêng của Nhóm (Mục 3.1), trong đó mỗi biến thể khác phiên bản liền trước đúng một thành phần (Mục 2-3).
2. **Triển khai một quy trình đánh giá tái lập được**: một bộ đo ghép cặp duy nhất, có kiểm định ý nghĩa thống kê, dùng chung cho cả sáu biến thể (Mục 4-5).
3. **So sánh với một baseline LLM trực tiếp** (V0: một lượt gọi LLM, không truy hồi, không đa tác tử) làm mốc cố định cho mọi phép so sánh, thay vì chỉ so các biến thể phức tạp với nhau (Mục 5).
4. **Phân tích đóng góp của từng thành phần** qua so sánh ghép cặp giữa các biến thể liền kề: trên MedQA, *cấu trúc lập luận đa tác tử*, chứ không phải *truy hồi* hay các lớp bộ nhớ/kiểm chứng thêm sau đó, mới là yếu tố làm thay đổi độ chính xác một cách có ý nghĩa thống kê (Mục 5-7).

2 Kiến trúc hệ thống

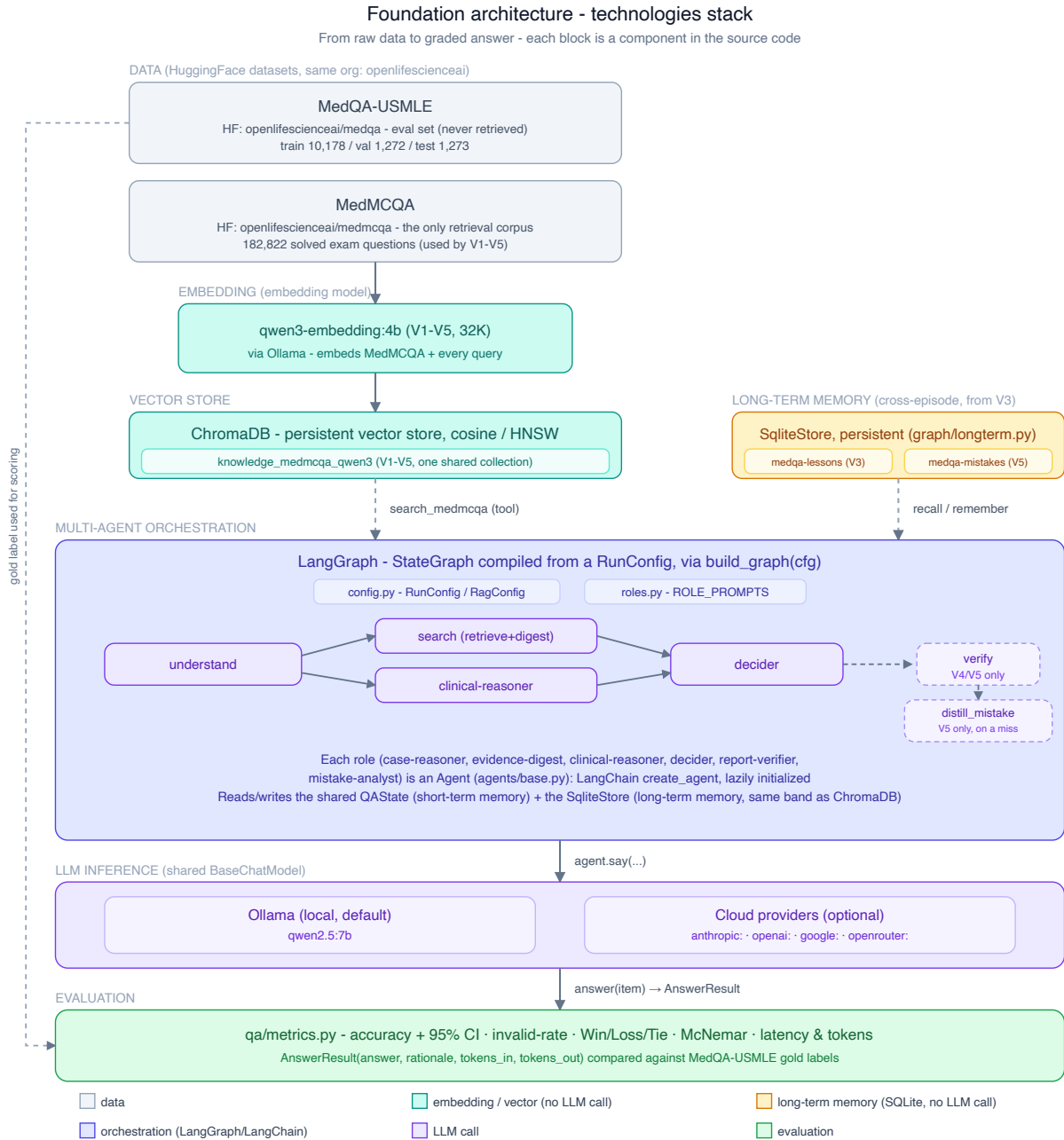
Ý tưởng là tạo một bộ khung ứng dụng duy nhất với các nút được bật/tắt tùy biến thể:

understand $\rightarrow$ (search || reasoning) $\rightarrow$ decide $\rightarrow$ [verify] $\rightarrow$ [distill_mistake]

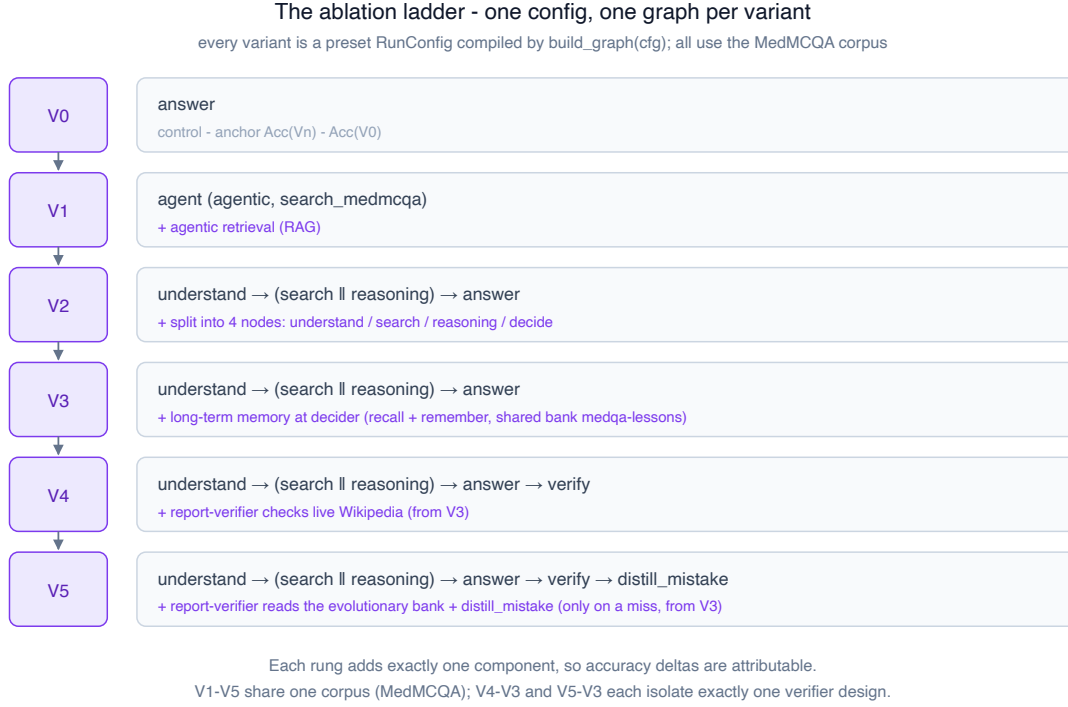
Một biến thể không được tạo bởi một nhánh mã riêng, mà nó phụ thuộc vào một `RunConfig` định sẵn, sau đó được biên dịch thành một `StateGraph` của `LangGraph` [4] qua `build_graph(cfg)`.

Vai trò từng thành phần trong stack (Hình 1).

- **Dữ liệu (HuggingFace datasets): MedQA-USMLE** [1] là bộ đề dùng để đánh giá, không bao giờ dùng để truy hồi; **MedMCQA** [7] là dữ liệu truy hồi duy nhất, dùng chung cho toàn bộ V1-V5.
- **Embedding**: một embedder duy nhất qua Ollama cho toàn bộ V1-V5 - **qwen3-embedding:4b** (ngữ cảnh 32K token). V2-V5 dùng một embedder ngữ cảnh ngắn hơn (8192 token) ở một thiết kế trước đó, nhưng đã chuyển sang cùng embedder với V1 sau một A/B nhỏ - xem §3.2.
- **Lưu trữ vector - ChromaDB**: vector persistent database, đo độ tương đồng bằng Cosine Estimation, một bộ sưu tập duy nhất (`knowledge_medmcqa_qwen3`), dùng chung cho V1-V5.
- **Bộ nhớ dài hạn (cross-episode) - SqliteStore**: persistent database cho nhiều round (`data/longterm/`, `graph/longterm.py`), tách biệt với `QASState` (bộ nhớ ngắn hạn trong một câu). Hai namespace độc lập: `medqa-lessons` - bài học chung mà `decider` tự recall/remember (từ V3), và `medqa-mistakes` - kho tiền hoá các ca đã trả lời SAI mà `verify` recall và `distill_mistake` remember (chỉ V5, §3.3).
- **Điều phối đa tác tử - LangGraph**: `StateGraph` biên dịch một `RunConfig` (`config.py`) thành đồ thị các nút (`graph/nodes.py`, `graph/build.py`); mọi nút đọc/ghi qua `QASState` dùng chung (`graph/state.py`) - đây chính là kênh giao tiếp chính của các nodes trong hệ thống (§3.3).



Hình 1: Technology stack: từ dữ liệu thô đến đáp án được chấm.



Hình 2: Thang ablation: mỗi biến thể thêm đúng một khả năng.

- **Tác tử - LangChain:** mỗi role trong đồ thị (tối đa 6 roles ở V5, xem Hình 1) là một **Agent** (`agents/base.py`), bọc `create_agent` của LangChain - khởi tạo không phụ thuộc nhà cung cấp mô hình LLM. Prompt của từng role nằm trong `roles.py`.
- **Suy luận LLM:** Ollama cục bộ (`qwen2.5:7b`) là mặc định; các nhà cung cấp đám mây (`anthropic:`, `openai:`, `google:`, `openrouter:`) gọi bằng interface `BaseChatModel` qua tham số `provider:model` (§4).
- **Đánh giá:** `qa/metrics.py` so `AnswerResult` (đáp án, giải thích, token) với nhãn đúng của MedQA-USMLE, tính độ chính xác kèm khoảng tin cậy bootstrap, tỷ lệ không hợp lệ, Thắng/Thua/Hòa, và McNemar (§5).

Bảng 1: Các nút thực sự được nối trong mỗi biến thể (tên nút khớp với mã nguồn).

Biến thể	Các nút tác tử	Số lượng tác tử
V0	<code>answer</code>	1
V1	<code>agent</code> (<code>agentic</code> , gọi <code>search_medmcqa</code>)	1
V2	<code>understand</code> → (<code>search</code> <code>reasoning</code>) → <code>answer</code>	4
V3	<code>understand</code> → (<code>search</code> <code>reasoning</code>) → <code>answer</code> + long-term memory	4
V4	V3's graph + <code>verify</code> (Wikipedia-grounded)	5
V5	V3's graph + <code>verify</code> + <code>distill_mistake</code>	6

V1 là ngoại lệ của bộ khung trên: thay vì tách `understand`/`search`/`reasoning` thành các nút riêng, nó rút về một nút **agent** duy nhất, tự làm cả bốn việc trong một lượt hội thoại có gọi công cụ (xem §3.2). V2-V5 theo đúng bộ khung bốn nút, với Node 1 (`understand`) sinh ra bản tóm

tất cả bệnh + truy vấn dùng chung cho hai nhánh Node 2 (**search**) và Node 3 (**reasoning**) chạy *đồng thời*, trước khi Node 4 (**answer**, vai **decider**) tổng hợp; V3 không thêm nút nào (**decider** tự đọc/ghi bộ nhớ dài hạn), V4/V5 mỗi biến thể thêm các nút sau **answer** (Node 5 thêm **verify**; V5 thêm cả Node 6 **verify** và **distill_mistake**).

3 Thiết kế hệ thống

3.1 Các biến thể: V0-V5

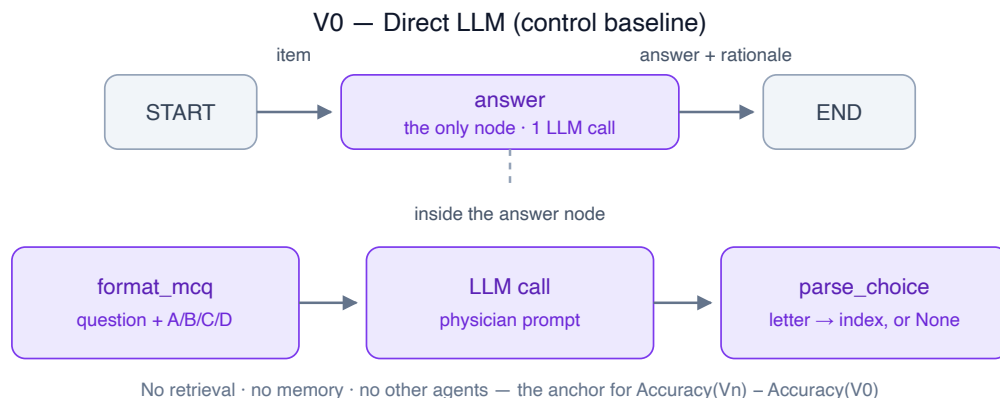
Mỗi tác tử là một lớp khởi tạo quanh `create_agent` của LangChain với một `BaseChatModel` không phụ thuộc nhà cung cấp. Mục này giới thiệu từng biến thể theo đúng thứ tự thang ablation - luồng xử lý, các tác tử tham gia, và điều mỗi layer thêm vào so với layer trước; §3.2 đi sâu hơn vào các quyết định thiết kế dùng chung cho V1-V5.

V0 - LLM trực tiếp (baseline). Một LLM duy nhất, không RAG: `format_mcq` dựng prompt (tình huống + A-D), model trả lời, `parse_choice` trích chữ cái. Đây là mốc mà mọi biến thể khác được so sánh vào: Độ chính xác(V_n) – Độ chính xác(V_0).

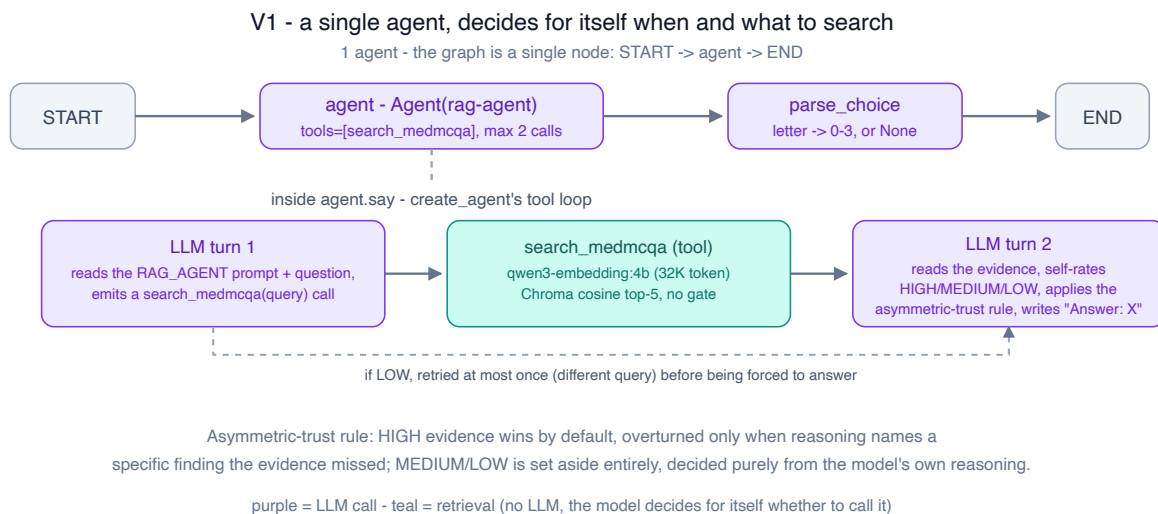
V1 - một tác tử agentic RAG. Đồ thị rút về một node: `START` → `agent` → `END`. Bên trong, `Agent(rag-agent)` tự quyết định có gọi `search_medmcqa` hay không (tối đa hai lần), tự chấm độ tin cậy của bằng chứng lấy về (HIGH/MEDIUM/LOW), rồi áp luật *tin cậy bất đối xứng* (§3.2) để chốt đáp án - tất cả trong cùng một tác tử, không có node/tác tử riêng cho từng bước.

V2 - bốn nút: hiểu ca bệnh, tìm kiếm, lập luận, quyết định. V2-V4 dùng chung một thiết kế bốn nút, được tách dựa trên bốn việc mà tác tử đơn của V1 làm bên trong nó, được tạo và kết nối thông qua LangGraph:

- **Node 1 - case-reasoner (hiểu ca bệnh):** đọc tình huống một lần, viết ra một bản tóm tắt ca bệnh và một truy vấn tìm kiếm, dùng cho hai nút sau.
- **Node 2 - search (tìm kiếm + rút gọn):** gọi `search_medmcqa` với truy vấn của Node 1, rồi `evidence-digest` tóm bằng chứng thành một đoạn ngắn kèm mức tin cậy tự đánh giá.
- **Node 3 - clinical-reasoner (lập luận):** lập luận từ bản tóm tắt của Node 1, hoàn toàn không thấy bằng chứng của Node 2 (hai nhánh chạy đồng thời); tự chấm mức tin cậy của



Hình 3: V0: một node, không RAG.



Hình 4: V1: một tác tử agentic tự gọi công cụ và tự quyết định.

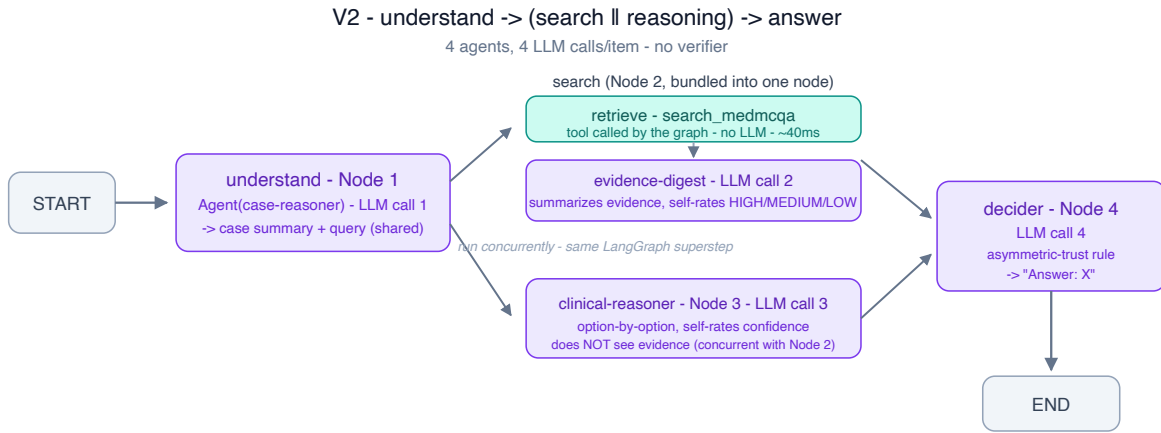
chính lập luận đó.

- **Node 4 - decider (quyết định):** nhận hai đầu vào độc lập kèm mức tin cậy riêng, áp luật tin cậy bất đối xứng để chốt đáp án.

V3 - V2 nhưng decider có bộ nhớ dài hạn. V3 giữ nguyên bốn nút của V2, nhưng bật `long_term=True`: Node 4 (decider) tự **recall** bài học từ các câu **TƯƠNG TỰ** đã gặp trước đó trước khi chốt đáp án, rồi tự **remember** lại một bài học sau khi chốt - không cần một tác tử riêng cho việc "nhớ", thông qua tool **remember/recall** mà **decider** sẵn có.

V4 - V3 + bộ kiểm chứng đối chiếu Wikipedia. V4 thêm **Node 5 - report-verifier**: đọc lại báo cáo lập luận của Node 3 và bản tóm tắt ca bệnh của Node 1 (`memory=True`), rồi xác nhận hoặc chỉnh đáp án của Node 4 (§3.4). Khi báo cáo không đủ để quyết, nó gọi tool **search_wikipedia** - một lệnh gọi Wikipedia *trực tuyến*, độc lập với dữ liệu MedMCQA cục bộ mà toàn hệ thống dùng.

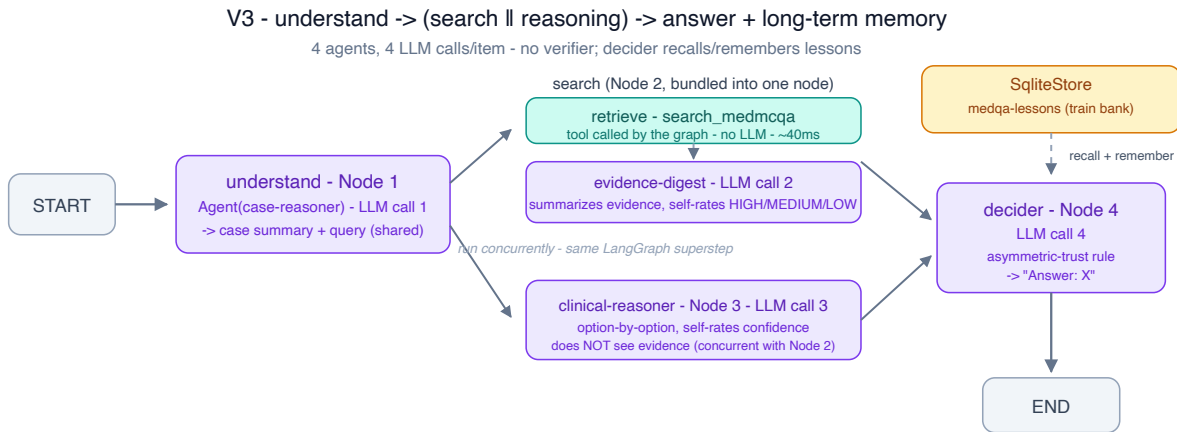
V5 - V3 + bộ kiểm chứng đọc dùng lại các ca đã sai. V5 thêm **CÙNG** Node 5 của V4 nhưng không gọi Wikipedia; thay vào đó nó **recall** từ một kho dữ liệu *riêng biệt* với kho chung của V3 - **kho tiến hoá (medqa-mistakes)**, chỉ chứa bài học từ những câu hệ thống từng trả lời SAI (§3.3). Bộ kiểm chứng không bao giờ biết đáp án đúng của câu đang xử lý - nó chỉ đọc bài học đã đúc kết từ những câu **KHÁC**, tương tự về chủ đề. V5 thêm **Node 6 - distill_mistake**, chạy SAU **verify**: đây là nút **DUY NHẤT** trong toàn đồ thị đọc đáp án đúng (`item.answer_idx`) - nếu đáp án cuối cùng SAI, nó gọi một tác tử **mistake-analyst** (được cho biết đáp án đúng) để đúc kết một bài học sửa sai, rồi ghi vào kho tiến hoá cho các câu tương lai; nếu đáp án **ĐÚNG**, nó không làm gì.



Node 2 and Node 3 never see each other's output; Node 4 applies an explicit rule, not free-form synthesis.
 Different from the multi-expert panel design (MedAgents) tried earlier - this is 4 steps of ONE pipeline, not several opinions debating.

purple = LLM call - teal = retrieval (no LLM, called directly by the graph - deterministic)

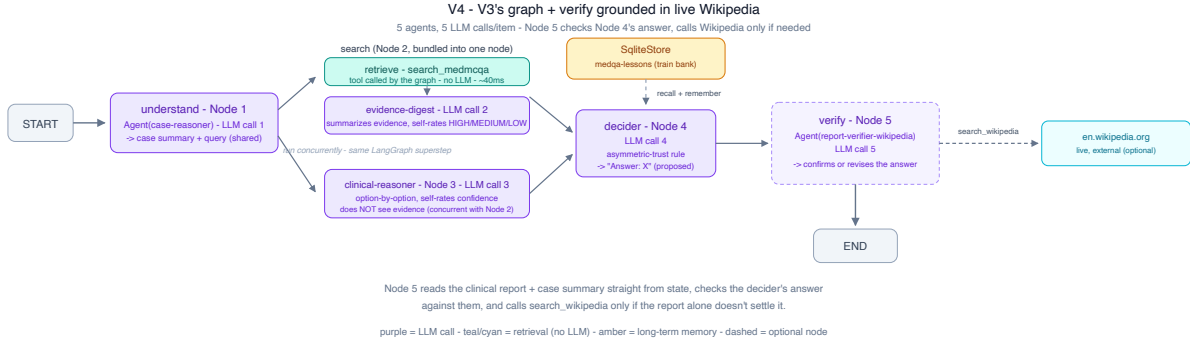
Hình 5: V2: Node 1 phân nhánh song song sang Node 2/3, hội tụ ở Node 4.



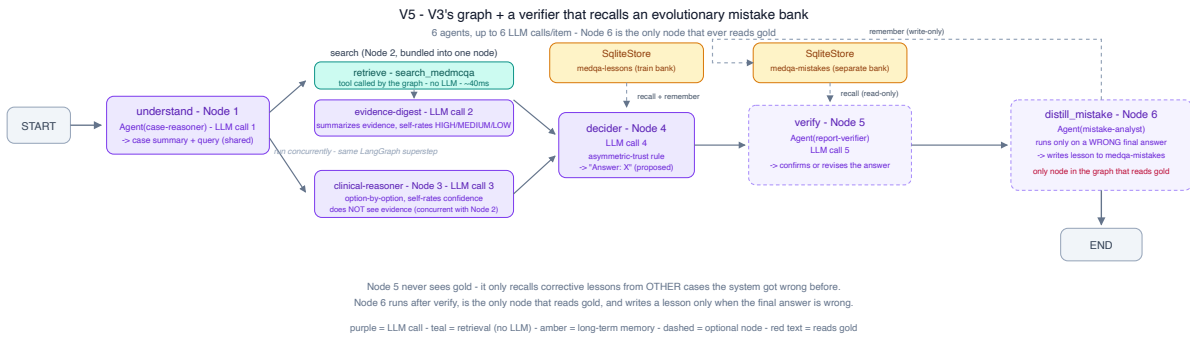
decider recalls lessons from similar earlier cases before answering, then writes one back -
 no separate agent or LLM call, just two store reads/writes around the existing decider call.

purple = LLM call - teal = retrieval (no LLM) - amber = long-term memory (SQLite, no LLM)

Hình 6: V3: bốn nút của V2; decider tự recall/remember vào medqa-lessons.



Hình 7: V4: đồ thị V3 + Node 5 (report-verifier-wikipedia), audit đáp án và gọi `search_wikipedia` khi cần.

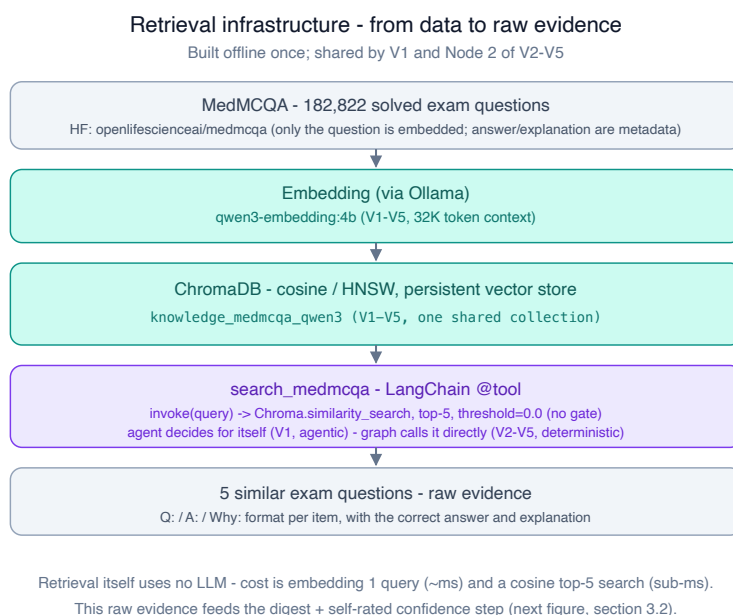


Hình 8: V5: đồ thị V3 + Node 5 (đọc medqa-mistakes) + Node 6 `distill_mistake`, chỉ ghi khi đáp án sai.

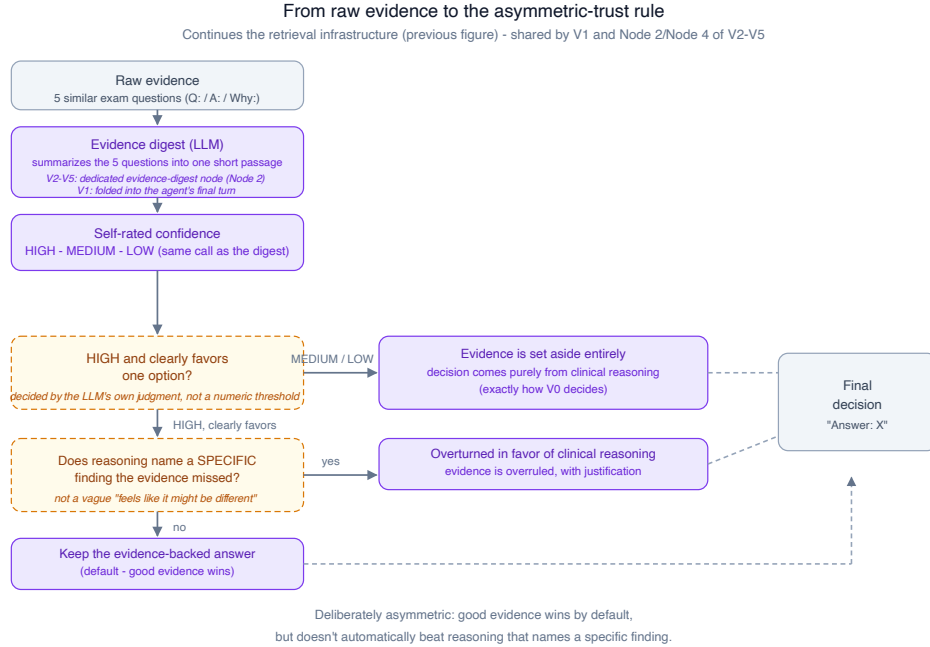
3.2 Thiết kế RAG

Toàn hệ thống dùng chung một *nguồn dữ liệu* và một *embedder* truy hỏi.

1. **Dữ liệu. MedMCQA** [7] (182.822 câu thi đã giải kèm đáp án và giải thích) là dữ liệu truy hỏi *duy nhất*, dùng cho toàn bộ V1-V5 - tức truy hỏi các ví dụ mẫu đã giải (tương tự Medprompt [6]), không phải văn bản tham khảo. Chỉ phần câu hỏi được nhúng; đáp án và giải thích nằm trong metadata.
2. **Embedder.** Một embedder duy nhất: `qwen3-embedding:4b` (ngữ cảnh 32K token, chạy qua Ollama), được áp dụng cho cả toàn bộ V2-V5.
3. **Vector Database.** ChromaDB, một collection duy nhất (`knowledge_medmcqa_qwen3`), do độ phù hợp bằng cosine similarity.
4. **Cơ chế truy hỏi.** `search_medmcqa` trả về $k=5$ câu thi gần nhất kèm đáp án và giải thích (định dạng Q:/A:/Why:).
5. **Đánh giá độ liên quan bằng nhãn.** Hệ thống tự đánh giá độ liên quan của bằng chứng vừa truy hỏi bằng một nhãn HIGH/MEDIUM/LOW, không so điểm cosine với một ngưỡng cố định ở bất kỳ đâu.
6. **Tối ưu truy vấn (query reformulation).** Nếu độ liên quan bị đánh giá LOW, viết lại một truy vấn tìm kiếm mới bằng cách thêm ngữ cảnh (context) từ tình huống, rồi truy hỏi lại.
7. **Luật quyết định: tin cậy bất đối xứng.** Một luật dùng chung cho bước chốt đáp án của V1 và Node 4 (decider, V2-V5). Nếu bằng chứng được chấm HIGH và nghiêng rõ về một phương án, phương án đó là đáp án mặc định; chỉ bị lật lại khi tình huống có một dấu hiệu *cụ thể* mà bằng chứng đã bỏ sót hoặc hiểu sai, không phải một cảm giác chung chung là “có lẽ khác”. Nếu bằng chứng chỉ MEDIUM/LOW hoặc không có, nó bị bỏ qua hoàn toàn và đáp án chọn thuần từ lập luận lâm sàng, đúng như cách V0 quyết định.



Hình 9: Hạ tầng truy hỏi: MedMCQA → embedding → ChromaDB → `search_medmcqa`.



Hình 10: Từ bằng chứng thô đến luật tin cậy bất đối xứng, dùng chung cho V1 và Node 4 (decider).

3.3 Thiết kế bộ nhớ

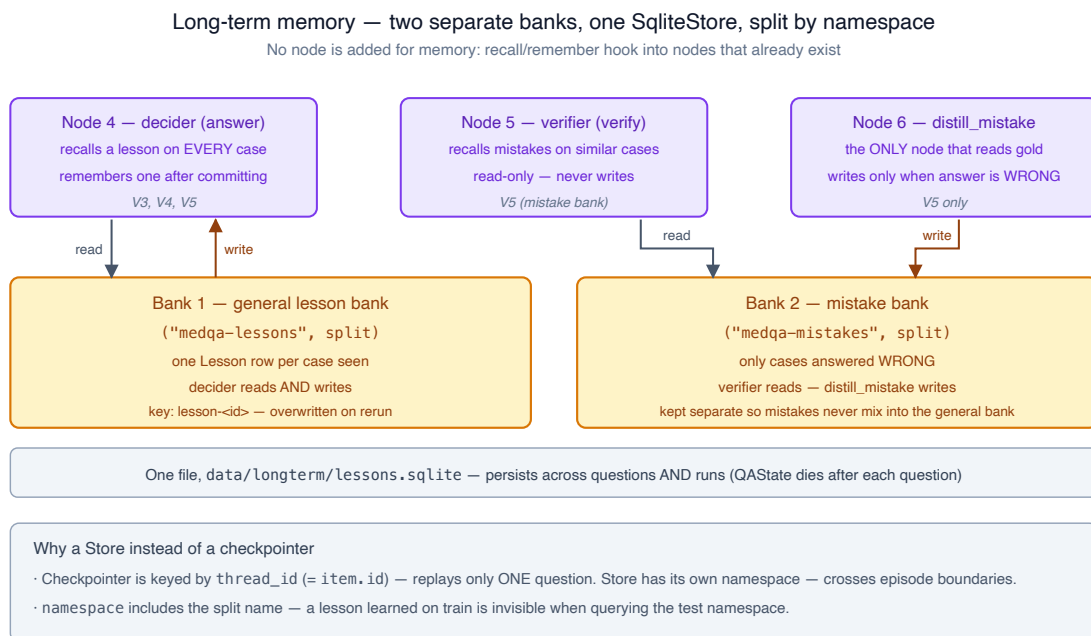
Bộ nhớ ngắn hạn (trong một câu hỏi). Dựa thẳng vào `state`: mỗi biến thể chạy trên cùng một `StateGraph(QAState)` (`graph/build.py`), và `QAState` (một `TypedDict`, `graph/state.py`) chính là bộ nhớ ngắn hạn - mỗi nút ghi ra một vài trường (`case_understanding`, `evidence`, `clinical_report`, ...), nút chạy sau đọc thẳng từ đó, không cần một tác tử/kho bộ nhớ riêng. Mặc định mọi biến thể (V0-V5) đều có nó: khác biệt giữa các biến thể chỉ là số nút, và do đó số trường tồn tại trong hệ thống. (Bảng 1).

Bảng 2: Short-term memory: each `QAState` field is a channel from one node to the next.

Trường trong <code>QAState</code>	Ghi bởi	Đọc bởi
<code>query</code>	Node 1 (understand)	Node 2 (search)
<code>evidence</code>	Node 2 (search)	Node 4 (decider), Node 5 (verifier)
<code>clinical_report</code>	Node 3 (reasoning)	Node 4 (decider), Node 5 (verifier)
<code>case_understanding</code>	Node 1 (understand)	Node 3 (reasoning), Node 5 (verifier)
<code>answer, rationale</code>	Node 4 (decider)	overwritten by Node 5, if present (V4/V5); Node 6 reads <code>answer</code> to check correctness (V5)

Bộ nhớ dài hạn (episodic, sử dụng qua nhiều câu hỏi): hai kho tách biệt. Cả hai dùng chung một `SqliteStore` của `LangGraph` (`data/longterm/lessons.sqlite`, `graph/longterm.py`), phân biệt bằng namespace:

- **Kho chung** (`medqa-lessons`, V3-V5): `decider` (Node 4) `recall` bài học từ những câu TUỞNG TỰ đã gặp trước - xếp hạng bằng số từ chủ đề trùng nhau (không phải embedding, để giữ chi phí thấp và không cần một collection thứ hai) - rồi `remember` một bài học sau khi



Hình 11: Bộ nhớ dài hạn: hai kho tách biệt bằng namespace trong một `SqliteStore`; `decider` đọc/ghi kho chung, bộ kiểm chứng chỉ đọc kho lỗi.

chốt, cho mọi câu, đúng lẫn sai.

- **Kho tiến hoá** (`medqa-mistakes`, `V5`): chỉ chứa bài học từ những câu hệ thống từng trả lời sai. Bộ kiểm chứng (Node 5) chỉ **recall** từ kho này, không bao giờ ghi và không bao giờ thấy đáp án đúng; Node 6 (`distill_mistake`), chạy sau Node 5, là nút duy nhất đọc đáp án đúng, và chỉ ghi một bài học khi đáp án cuối cùng sai (§3.1).

Chống rò rỉ dữ liệu (contamination). Rủi ro là: bài học của câu thứ N đang được chấm rò sang câu thứ $N+k$ trong CÙNG lượt chấm đó, làm tăng ảo độ chính xác vì lý do không liên quan tới chất lượng lập luận. Lớp bảo vệ đang có là *namespace*: mỗi kho gồm cả tên *split*, nên bài học học trên **train** là vô hình khi truy vấn namespace **test**.

3.4 Thiết kế bộ kiểm chứng

`V4` và `V5` dùng chung một Node 5 (**report-verifier**), chạy ngay sau khi **decider** chốt đáp án. Hai biến thể chỉ khác nhau ở một điểm: nguồn bằng chứng bổ sung mà Node 5 tra cứu khi báo cáo của Node 3 chưa đủ rõ để quyết.

- **V4 - tra Wikipedia trực tuyến.** Gọi công cụ `search_wikipedia` qua MediaWiki API, có timeout; nếu mạng lỗi, nó trả về chuỗi rỗng và hệ thống suy biến về đúng hành vi của `V3`, chứ không làm hỏng lượt chạy. Đây là nguồn bằng chứng độc lập đầu tiên trong toàn hệ thống không bắt nguồn từ MedMCQA. Chọn Wikipedia thay vì tìm kiếm chung trên internet vì Wikipedia chỉ có nội dung bách khoa về y khoa (bệnh, thuốc, cơ chế...), không thể vô tình lộ đáp án của một câu hỏi thi cụ thể như một trang đề thi hay diễn đàn giải đề có thể làm.
- **V5 - tra lại kho các câu từng trả lời sai.** Gọi `recall_mistakes` để đọc từ kho `medqa-mistakes` (§3.3) bài học rút ra từ những câu tương tự mà hệ thống từng trả lời sai trước đó. Bộ kiểm chứng ở đây không có công cụ tra cứu nào khác ngoài kho này, và chỉ đọc: nó không bao giờ ghi, và không bao giờ thấy đáp án đúng của câu đang xử lý.

V5 có thêm một nút mà V4 không có: **Node 6 - distill_mistake**, chạy ngay sau Node 5. Đây là nút duy nhất trong toàn đồ thị được phép đọc đáp án đúng (`item.answer_idx`). Nếu đáp án cuối cùng đúng, nó không làm gì cả, không tốn thêm lượt gọi LLM nào. Nếu sai, nó gọi riêng một tác tử (**mistake-analyst**), cho biết cả đáp án hệ thống vừa chọn lẫn đáp án đúng, để rút ra một bài học sửa sai rồi ghi vào kho tiền hoá cho những câu hỏi khác đọc lại về sau. Nút này không bao giờ được phép sửa đáp án của câu đang xử lý - việc của nó chỉ là để lại bài học cho lần sau, không phải ảnh hưởng điểm số của câu hiện tại.

4 Thiết lập thực nghiệm

4.1 Cấu trúc mã nguồn

Toàn bộ mã nguồn nằm dưới `src/agent_hospital/`, chia theo nhiệm vụ - mỗi thư mục con một mối quan tâm riêng (đồ thị LangGraph, tác tử, truy hồi, đánh giá):

<code>src/agent_hospital/</code>	
<code>__main__.py</code>	CLI: chạy một biến thể trên MedQA-USMLE, in ra metric
<code>config.py</code>	<code>RunConfig/RagConfig</code> - nơi tập trung mọi tùy chỉnh
<code>models.py</code>	<code>provider:model -> BaseChatModel</code> , không phụ thuộc nhà cung cấp
<code>roles.py</code>	registry system prompt (9 vai, xem bảng tóm tắt ở 4.2)
<code>predict.py</code>	chạy một biến thể, ghi dự đoán từng câu ra <code>.jsonl</code>
<code>evaluate.py</code>	tính metric từ các file dự đoán
<code>agents/</code>	
<code>base.py</code>	Agent - lớp bọc <code>create_agent</code> của <code>LangChain</code> , khởi tạo tree
<code>graph/</code>	
<code>state.py</code>	<code>QASState</code> - trạng thái dùng chung truyền qua mọi node
<code>build.py</code>	<code>RunConfig -> StateGraph</code> đã compile
<code>nodes.py</code>	các node dùng lại được (<code>state -> partial state</code>)
<code>longterm.py</code>	bi nhớ dài hạn: hai kho trong một <code>SqliteStore</code>
<code>qa/</code>	
<code>variants.py</code>	các preset V0-V5 + điem vào <code>build_variant</code>
<code>mcq.py</code>	dùng prompt cho câu hỏi, trích đáp án
<code>metrics.py</code>	đo lường từng câu + so sánh giữa biến thể
<code>reasoning.py</code>	tác tử rút gọn truy vấn (V1)
<code>knowledge/</code>	
<code>embeddings.py</code>	ham embedding cho kho tri thức RAG (Ollama)
<code>ingest.py</code>	nạp MedMCQA vào Chroma
<code>retriever.py</code>	truy hồi cosine top-k trên Chroma
<code>wikipedia.py</code>	tim Wikipedia trực tuyến (nguồn của bộ kiểm chứng V4)
<code>diseases/</code>	
<code>medqa_usmle.py</code>	nạp tập MedQA-USMLE (split dùng để chấm điểm)

Quy ước xuyên suốt: mỗi `RunConfig` (`config.py`) là toàn bộ tham số của một lượt chạy; `qa/variants.py` chỉ định nghĩa SÁU preset cụ thể (V0-V5) từ đúng một `RunConfig`, không có nhánh rẽ nào khác trong `graph/build.py` ngoài những cờ mà `RunConfig` khai báo.

4.2 Mô hình và prompt

Mô hình trả lời mặc định là **Qwen2.5** [8] (Ollama, có gọi công cụ): **qwen2.5:7b** cho mọi biến thể và mọi con số được báo cáo. Mọi vai chạy ở **temperature 0**: ở temperature mặc định, chênh lệch

V1 – V0 dao động giữa các lần chạy lớn hơn cả bản thân hiệu ứng, nên tính tất định là bắt buộc để McNemar ghép cặp có ý nghĩa. Các nhà cung cấp đám mây (`anthropic:`, `openai:`, `google:`, `openrouter:`) được hỗ trợ qua cùng một đặc tả `provider:model`.

Toàn hệ thống có 9 vai (`roles.py:ROLE_PROMPTS`), mỗi vai một system prompt riêng, dùng nguyên văn tiếng Anh (prompt vận hành thực tế). Bảng 3 tóm tắt việc của từng vai; nội dung đầy đủ nằm trong mã nguồn. Node 1-4 xuất hiện ở mọi biến thể V2-V5 (§3.1).

Bảng 3: Tóm tắt system prompt của 9 vai trong hệ thống.

Vai (<code>roles.py</code>)	Dùng ở	Tóm tắt
<code>baseline</code>	V0	Trả lời trực tiếp từ kiến thức riêng.
<code>rag-agent</code>	V1	Tự hiểu ca, tìm kiếm, lập luận, quyết định.
<code>case-reasoner</code>	Node 1	Tóm ca bệnh, viết truy vấn tìm kiếm.
<code>clinical-reasoner</code>	Node 3	Lập luận option-by-option, tự chấm tin cậy.
<code>evidence-digest</code>	Node 2	Tóm bằng chứng, tự chấm tin cậy.
<code>decider</code>	Node 4	Tổng hợp báo cáo + bằng chứng, chốt đáp án.
<code>report-verifier</code>	Node 5 (V5)	Soát đáp án; tra thêm kho câu từng sai.
<code>report-verifier-wikipedia</code>	Node 5 (V4)	Soát đáp án; tra thêm Wikipedia.
<code>mistake-analyst</code>	Node 6 (V5)	Chỉ chạy khi sai: đúc kết một bài học.

`parse_choice` trích đáp án từ **Answer: X** hoặc **answer is X** (không phân biệt hoa thường), dự phòng bằng chữ cái A|B|C|D độc lập đầu tiên; phản hồi không trích được trả về `None` và bị tính là *không hợp lệ*, không bao giờ bị tính nhầm thành câu trả lời sai. Tỷ lệ không hợp lệ đo được là **0.0%** xuyên suốt.

4.3 Phần cứng / thiết lập API

- **Phần cứng:** chạy trên một máy cá nhân, chạy các dịch vụ cục bộ.
- **Dữ liệu:** `openlifescienceai/medqa` (train 10,178 / validation 1,272 / **test 1,273**), lưu cache cục bộ và nạp không cần mạng.

Lệnh `predict` và `evaluate` cho từng biến thể:

```
PYTHONPATH=src .venv/bin/python -m agent_hospital.predict -v V0 -s test -m qwen2.5:7b
PYTHONPATH=src .venv/bin/python -m agent_hospital.predict -v V1 -s test -m qwen2.5:7b
PYTHONPATH=src .venv/bin/python -m agent_hospital.predict -v V2 -s test -m qwen2.5:7b
PYTHONPATH=src .venv/bin/python -m agent_hospital.predict -v V3 -s test -m qwen2.5:7b
PYTHONPATH=src .venv/bin/python -m agent_hospital.predict -v V4 -s test -m qwen2.5:7b
PYTHONPATH=src .venv/bin/python -m agent_hospital.predict -v V5 -s test -m qwen2.5:7b

PYTHONPATH=src .venv/bin/python -m agent_hospital.evaluate --dir data/eval_runs --baseline V0
```

`predict.py` ghi dự đoán từng câu vào `data/eval_runs/<variant>_<split>_<model>.jsonl` kèm một tệp `.meta.json` (cấu hình lượt chạy); chạy lại cùng lệnh sẽ RESUME - bỏ qua những câu đã có trong tệp, chỉ chấm tiếp các câu còn thiếu, an toàn nếu lượt chạy trước bị ngắt giữa chừng. `evaluate.py` đọc mọi tệp `.jsonl` trong `--dir` và in ra bảng leaderboard (độ chính xác, khoảng tin cậy bootstrap, tỷ lệ không hợp lệ) cùng bảng so sánh ghép cặp với `--baseline`.

5 Kết quả đánh giá

Chỉ số chính là độ chính xác, Độ chính xác = (số dự đoán đúng)/(số câu hỏi). Nhóm báo cáo thêm tỷ lệ phản hồi không hợp lệ, mức tăng độ chính xác so với V0, Thắng/Thua/Hòa so với baseline, khoảng tin cậy bootstrap 95%, kiểm định ghép cặp McNemar, độ trễ và số token.

Kết quả dưới đây đo trên **600 câu đầu của tập test** ($n=600/1273$, qwen2.5:7b, temperature 0) cho cả sáu biến thể V0-V5 - cùng 600 câu, cùng thứ tự (xác nhận bằng `item_id`), nên so sánh chéo giữa mọi biến thể đều hợp lệ.

5.1 Bảng chính (main board): 600 câu đầu của tập test, qwen2.5:7b

Bảng 4: Kết quả trên 600 câu đầu của tập test, qwen2.5:7b, temperature 0.

Variant	Accuracy	95% Bootstrap CI	Invalid	Latency/query	Tokens/query
V0: LLM trực tiếp	0.602 (361/600)	[0.562, 0.640]	0.0%	1.35 s	497
V1: RAG-only (agen-tic)	0.593 (356/600)	[0.555, 0.635]	0.3%	4.80 s	2877
V2: Đa tác tử (4 nút)	0.657 (394/600)	[0.618, 0.695]	0.0%	19.51 s	4428
V3: V2 + bộ nhớ dài hạn	0.653 (392/600)	[0.617, 0.692]	0.0%	21.29 s	4633
V4: V3 + bộ kiểm chứng Wikipedia	0.635 (381/600)	[0.597, 0.675]	0.0%	25.02 s	6423
V5: V3 + bộ kiểm chứng kho tiến hoá	0.637 (382/600)	[0.598, 0.675]	0.0%	25.97 s	6554

Tỷ lệ không hợp lệ là 0.0% ở năm biến thể; V1 có 2 câu không hợp lệ trên 600 (0.3%) - vòng lặp gọi công cụ đôi khi không hội tụ về một chữ cái. V1 thấp hơn V0 một chút (-0.8 điểm, không có ý nghĩa thống kê - xem McNemar dưới), nhất quán với mọi lần đo retrieval-only trong dự án: truy hồi riêng không giúp ích trên MedQA. V2 **cao nhất** ($+5.5$ điểm so với V0); V3 đứng ngay sau ($+5.2$ điểm) - cả hai đều có ý nghĩa thống kê (xem McNemar dưới); V4 và V5 thấp hơn một chút ($+3.3$ và $+3.5$ điểm), cao hơn V0 nhưng chưa đạt ngưỡng ý nghĩa thống kê.

5.2 So sánh với baseline (so sánh ghép cặp)

5.3 Phân tích Thắng/Thua/Hòa

Cột T/T/H đếm theo từng câu: biến thể sửa được lỗi của baseline (thắng), làm hỏng một câu baseline vốn đúng (thua), hay trùng khớp (hòa). **V2 vs V0**: 86 thắng so với 53 thua, và với 139 câu bất đồng, McNemar cho $p=0.0064$: cải thiện có ý nghĩa thống kê. **V3 vs V0**: mẫu hình tương tự - 87 thắng so với 56 thua, $p=0.0118$ - cũng có ý nghĩa thống kê: bộ nhớ dài hạn giữ được mức cải thiện mà V2 tạo ra, không làm mất đi. **V1 vs V0**: 71/76, gần như cân bằng - truy hồi sửa một số câu nhưng làm hỏng nhiều hơn, ròng lại là một khoản lỗ nhỏ không đáng kể. **V4, V5 vs V0**: cùng mẫu hình thắng nhiều hơn thua (80/60, 81/60) nhưng chưa đạt ngưỡng ý nghĩa thống kê ($p=0.108$ và $p=0.092$). **So với V2**: cả ba biến thể thêm vào sau (V3, V4, V5) đều thấp hơn một chút, không có khác biệt ý nghĩa thống kê (mọi $p \geq 0.06$, gần nhất là V4 vs V2 với $p=0.060$) - ở quy mô $n=600$ này, vẫn chưa có bằng chứng rằng bộ nhớ dài hạn hay một trong hai thiết kế bộ kiểm chứng cải thiện thêm so với riêng kiến trúc đa tác tử của V2. Đây là phát hiện ngược với kỳ vọng ban đầu của Nhóm, và được báo cáo trung thực thay vì diễn giải có lợi cho thiết kế.

Bảng 5: So sánh ghép cặp trên 600 câu đầu của tập test, gwen2.5:7b. Tính trực tiếp từ bản ghi từng câu.

Comparison	Delta	W/L/T	McNemar p
V1 vs V0	-0.8 điểm	71/76/453	0.7416 (không ý nghĩa)
V2 vs V0	+5.5 điểm	86/53/461	0.0064 (có ý nghĩa)
V3 vs V0	+5.2 điểm	87/56/457	0.0118 (có ý nghĩa)
V4 vs V0	+3.3 điểm	80/60/460	0.1080 (không ý nghĩa)
V5 vs V0	+3.5 điểm	81/60/459	0.0918 (không ý nghĩa)
V3 vs V2 (hiệu ứng bộ nhớ dài hạn)	-0.3 điểm	8/10/582	0.8145 (không ý nghĩa)
V4 vs V2 (hiệu ứng kiểm chứng Wikipedia)	-2.2 điểm	14/27/559	0.0596 (không ý nghĩa)
V5 vs V2 (hiệu ứng kiểm chứng kho tiến hoá)	-2.0 điểm	14/26/560	0.0807 (không ý nghĩa)
V4 vs V3 (tách riêng đúng một bộ kiểm chứng)	-1.8 điểm	14/25/561	0.1081 (không ý nghĩa)
V5 vs V3 (tách riêng đúng một bộ kiểm chứng)	-1.7 điểm	12/22/566	0.1214 (không ý nghĩa)

5.4 Chi phí và độ trễ

Chi phí đo bằng *độ trễ* và *số token* (đầu vào + đầu ra) cho mô hình cục bộ (không có phí API). V0: **1.35s/câu, 497 token/câu**. V1 (agentic): **4.80s/câu** ($\sim 3.6 \times V0$), **2877 token/câu** ($\sim 5.8 \times V0$) - vòng lặp gọi công cụ sinh nhiều token hơn một lượt trả lời trực tiếp. V2 (4 nút): **19.51s/câu** ($\sim 14 \times V0$), **4428 token/câu** - bốn tác tử, một số nút chạy tuần tự, nên chi phí cộng dồn theo số lượt gọi LLM. V3 (V2 + bộ nhớ dài hạn, vẫn 4 nút): **21.29s/câu, 4633 token/câu** - chậm hơn V2 dù không thêm nút hay lượt gọi LLM nào, chỉ hai lần đọc/ghi kho `SqliteStore` thêm quanh lượt gọi `decider` sẵn có. V4 (V3 + bộ kiểm chứng Wikipedia): **25.02s/câu** (~ 3.7 s hơn V3), **6423 token/câu** - chi phí của Node 5, gồm cả những lần thất sự gọi `search_wikipedia`. V5 (V3 + bộ kiểm chứng kho tiến hoá): **25.97s/câu, 6554 token/câu** - thêm cả Node 5 lẫn Node 6 (`distill_mistake`, chỉ tốn lượt gọi LLM khi đáp án cuối cùng sai). Bộ nhớ dài hạn (V3) đã cho thấy cải thiện có ý nghĩa thống kê so với V0 (xem trên), nên chi phí thêm của nó so với V2 ($\sim 9\%$ độ trễ, $\sim 5\%$ token) có cơ sở để giữ lại; hai thiết kế bộ kiểm chứng (V4, V5) thì chưa - chi phí thêm $\sim 28\text{-}33\%$ độ trễ và $\sim 45\text{-}48\%$ token của chúng so với V2 hiện chưa được biện minh bằng bằng chứng.

6 Phân tích lỗi

Mục này dùng tập dự đoán từng câu của V0-V2 (`data/eval_runs/*.jsonl`, $n=600$, xem §5) - không còn chỉ là mô tả định tính.

Tỷ lệ không hợp lệ là 0.0% ở V0 và V2, và 0.3% (2/600) ở V1, nên gần như mọi lỗi đều là chọn sai lựa chọn thực sự, không phải lỗi trích xuất. Hai mẫu hình hiện rõ trong dữ liệu ghép cặp:

- **Câu trung tính với truy hồi.** V1 vs V0: 71 thắng, 76 thua, 453 hòa (§5) - trên đa số câu (76%), truy hồi không đổi đáp án, nhất quán với việc MedQA chủ yếu đòi hỏi lập luận chứ không phải tra cứu dữ kiện.
- **Câu nhạy với cấu trúc lập luận.** V2 vs V1: 101 thắng, 63 thua (§5) - tách hiểu ca bệnh / tìm kiếm / lập luận / quyết định thành bốn nút làm đổi quyết định trên nhiều câu hơn hẳn so với chênh lệch do riêng truy hồi gây ra, khu trú phần lớn mức tăng vào khâu cấu trúc lập luận chứ không phải bằng chứng truy hồi (vẫn cùng một nguồn MedMCQA).

Bốn ví dụ trên minh hoạ hai mẫu hình lỗi lặp lại trong dữ liệu ghép cặp: V2 đôi khi lập luận

Bảng 6: Phân tích lỗi với ví dụ thật từ tập dự đoán $n=600$ (V0 vs V2).

Loại lỗi	Mã câu	Ghi chú
V2 lập luận chi tiết nhưng chốt sai, trong khi V0 (một lượt) tình cờ đúng	test-00012	V0 chọn đúng (B); Node 3 của V2 xây một chuỗi lập luận cụ thể dẫn tới u xơ thần kinh loại 2, decider theo lập luận đó và chọn D (sai).
Bị phương án nghe hợp lý nhưng sai đánh lừa	test-00013	V0 chọn đúng (D, sa van hai lá); V2 lập luận nghiêng về bệnh cơ tim phì đại tắc nghẽn (HOCM) và chọn A (sai) - một chẩn đoán phân biệt hợp lý nhưng không khớp đặc điểm tiếng thổi trong đề bài.
V2 sửa đúng nhờ tri thức chuyên biệt hơn	test-00003	V0 chọn sai (B, nhiễm khuẩn vùng chậu); Node 3 của V2 xác định đúng đặc điểm vi khuẩn học (khuẩn lạc hồng trên môi trường MacConkey) và chọn D (đúng).
V2 sửa đúng nhờ bám sát đúng câu hỏi được hỏi	test-00024	V0 chọn sai (B, tràn khí màng phổi áp lực); V2 nhận ra câu hỏi thực sự là bước xử trí tiếp theo chứ không phải chẩn đoán, chọn A (đặt nội khí quản, đúng).

chi tiết nhưng chốt sai trong khi V0 tình cờ đúng (hoặc bị một phương án nghe hợp lý đánh lừa), và V2 đôi khi sửa đúng nhờ tri thức chuyên biệt hơn hoặc bám sát đúng câu hỏi được hỏi hơn V0.

7 Thảo luận

- **Không dễ để xây dựng RAG hay bộ nhớ ngoài thật sự hữu ích.** Ba nỗ lực bổ sung ngữ cảnh ngoài mô hình trong khảo sát này - truy hồi (V1), bộ nhớ dài hạn (V3), bộ kiểm chứng tra cứu thêm (V4, V5) - đều KHÔNG cải thiện có ý nghĩa thống kê so với biến thể ngay trước nó (V3 vs V2, V4 vs V3, V5 vs V3 đều không ý nghĩa). Chỉ riêng việc tổ chức lại cấu trúc lập luận (V2, không thêm bằng chứng hay bộ nhớ nào mới) mới tạo ra khác biệt so với biến thể liền trước nó.
- **Đòn bẩy thật sự là chia việc lập luận ra cho các tác tử** V2 tách hiểu ca bệnh / tìm kiếm / lập luận / quyết định thành bốn nút, trên *cùng* bằng chứng truy hồi mà V1 đã có, và tạo ra mức cải thiện có ý nghĩa thống kê đầu tiên trong toàn bộ khảo sát (+5.5 điểm so với V0, $p=0.0064$; +6.3 điểm so với V1, $p=0.0037$). Bộ nhớ dài hạn (V3) giữ được mức cải thiện đó so với V0 (+5.2 điểm, $p=0.0118$), nhưng không thêm được gì so với chính V2 (xem dưới) - cải thiện của thang biến thể này đến từ V2, không phải từ các lớp thêm sau đó.
- **Kiến trúc modular mở đường cho các nghiên cứu về bảo mật hay tối ưu hiệu suất.** Mỗi thành phần (truy hồi, bộ nhớ, bộ kiểm chứng) là một node riêng, bật/tắt độc lập qua `build_variant` (§4.1) mà không phải viết lại pipeline. Cấu trúc này thuận lợi cho việc tối ưu hiệu suất từng node riêng lẻ (ví dụ giảm token/độ trễ của bộ kiểm chứng mà không đổi các node khác), cũng như cho nghiên cứu bảo mật sau này (thử các đầu vào đối kháng nhắm vào một node cụ thể, chẳng hạn prompt injection qua bằng chứng truy hồi hoặc qua bộ nhớ dài hạn, và cô lập ảnh hưởng trong phạm vi node đó).

8 Hạn chế

- **Bộ nhớ dài hạn (V3) chưa cho thấy lợi ích thêm so với V2.** V3 vs V2: -0.3 điểm, không ý nghĩa thống kê ($p=0.8145$, chỉ 18 câu bất đồng trên 600), dù decider tốn thêm hai lần đọc/ghi kho `SqliteStore` mỗi câu.

- **Cả hai thiết kế bộ kiểm chứng (V4, V5) cũng chưa cho thấy lợi ích thêm - dù tổn thêm chi phí đáng kể.** V4 vs V2: -2.2 điểm ($p=0.0596$, sát ngưỡng); V5 vs V2: -2.0 điểm ($p=0.0807$), cả hai chưa đạt ý nghĩa thống kê, trong khi tổn thêm $\sim 28-33\%$ độ trễ và $\sim 45-48\%$ token so với V2.
- **Chỉ thực nghiệm trên một mô hình, ở temperature 0.** Toàn bộ số liệu trong báo cáo đều chạy trên qwen2.5:7b, temperature 0 (§4); Nhóm chưa kiểm tra các phát hiện này có giữ nguyên trên mô hình khác (kích thước khác, nhà cung cấp khác) hay ở temperature khác 0 hay không.

9 Kết luận

Nhóm đặt ra bốn mục tiêu về thiết kế hệ thống, và cả bốn đều đạt được trên MedQA-USMLE. **Một kiến trúc đa tác tử dạng module:** một thang biến thể sáu bậc (ablation ladder) điều khiển hoàn toàn bằng cấu hình, trong đó mỗi bậc khác bậc liền trước đúng một thành phần, trên cùng một đồ thị nền LangGraph. **Một quy trình đánh giá tái lập được:** một bộ đo ghép cặp duy nhất, có nhận thức về ý nghĩa thống kê (bootstrap CI, McNemar), chấm cả sáu biến thể theo cùng một chuẩn. **Một baseline LLM trực tiếp:** V0 làm mốc cố định, để mọi thành phần thêm vào - dù nhỏ hay lớn - đều được so vào cùng một điểm xuất phát. **Phân tích đóng góp của từng thành phần:** bằng chứng đo được ở $n=600$ rõ ràng ở điểm cốt lõi - riêng truy hồi không cải thiện độ chính xác MedQA (V1 vs V0: -0.8 điểm, không ý nghĩa), trong khi tách cấu trúc lập luận thành bốn nút thì có (V2 vs V0: $+5.5$ điểm, $p=0.0064$), và bộ nhớ dài hạn (V3) giữ được mức cải thiện đó so với V0 ($+5.2$ điểm, $p=0.0118$). Nhưng không biến thể nào thêm vào trên V2 - kể cả V3 - cải thiện có ý nghĩa thống kê so với chính V2 (V3/V4/V5 vs V2: -0.3 đến -2.2 điểm), dù V4 và V5 tổn thêm đáng kể độ trễ và token. Giá trị chính của khảo sát không nằm ở một con số độ chính xác cuối cùng, mà ở chính quy trình: mỗi thành phần kiến trúc được cô lập, đo, và so sánh trên cùng một chuẩn - nên phát hiện phản trực giác (bộ nhớ và bộ kiểm chứng chưa thêm được gì so với V2) đáng tin cậy như phát hiện thuận trực giác (cấu trúc lập luận đa tác tử giúp ích).

Tài liệu tham khảo

- [1] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. volume 11, page 6421, 2021.
- [2] Qiao Jin, Won Kim, Qingyu Chen, Donald C Comeau, Lana Yeganova, W John Wilbur, and Zhiyong Lu. Medcpt: Contrastive pre-trained transformers with large-scale pubmed search logs for zero-shot biomedical information retrieval. *Bioinformatics*, 39(11), 2023.
- [3] Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik Siu Chan, Xuhai Xu, Daniel McDuff, Hyeon-hoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, and Hae Won Park. Mdagents: An adaptive collaboration of llms for medical decision-making. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.15155.
- [4] LangChain. Langgraph: Building stateful, multi-actor applications with llms. <https://langchain-ai.github.io/langgraph/>, 2024.
- [5] Junkai Li, Yunghwei Lai, Weitao Li, Jingyi Ren, Meng Zhang, Xinhui Kang, Siyu Wang, Peng Li, Ya-Qin Zhang, Weizhi Ma, and Yang Liu. Agent hospital: A simulacrum of hospital with evolvable medical agents. *arXiv preprint arXiv:2405.02957*, 2024.

- [6] Harsha Nori, Yin Tat Lee, Sheng Zhang, Dean Carignan, Richard Edgar, et al. Can generalist foundation models outcompete special-purpose tuning? case study in medicine. *arXiv preprint arXiv:2311.16452*, 2023.
- [7] Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Conference on Health, Inference, and Learning (CHIL)*, 2022.
- [8] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.
- [9] Xiangru Tang, Anni Zou, Zhuosheng Zhang, Ziming Li, Yilun Zhao, Xingyao Zhang, Arman Cohan, and Mark Gerstein. Medagents: Large language models as collaborators for zero-shot medical reasoning. In *Findings of the Association for Computational Linguistics (ACL Findings)*, 2024. arXiv:2311.10537.
- [10] Guangzhi Xiong, Qiao Jin, Zhiyong Lu, and Aidong Zhang. Benchmarking retrieval-augmented generation for medicine. *Findings of the Association for Computational Linguistics (ACL)*, 2024.